

Published as a conference paper at ICLR 2016

PRIORITIZED EXPERIENCE REPLAY

Tom Schaul, John Quan, Ioannis Antonoglou and David Silver

Google DeepMind

{schaul, johnquan, ioannisa, davidsilver}@google.com

Earlier We Introduced

- Experience Replay to break temporal correlations
 - Training assumes independent samples
 - We want the weight updates to happen on independent samples
- Replay buffer can store relatively rare experiences

Replaying Experiences

- Mnih et al. chose experiences from the replay buffer uniformly and randomly
 - Transitions in the buffer are replayed *with the same frequency* as experienced by the agent
- Instead replay transitions with the goal of improving learning progress

Recall: Algorithm (Mnih et al.)

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

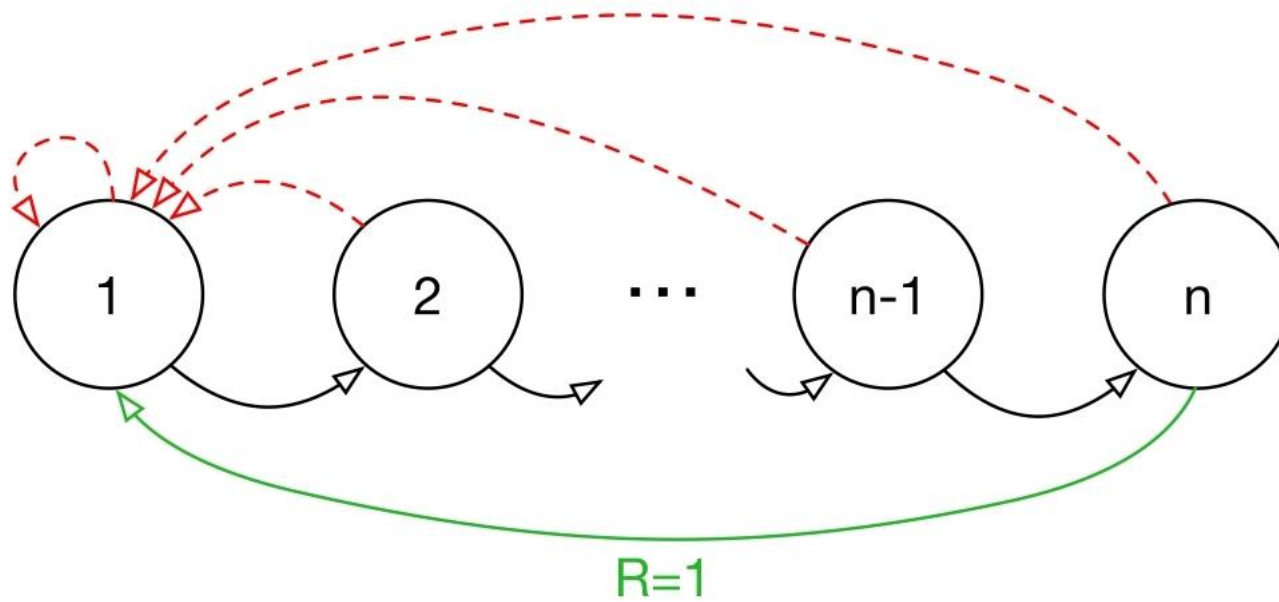
 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

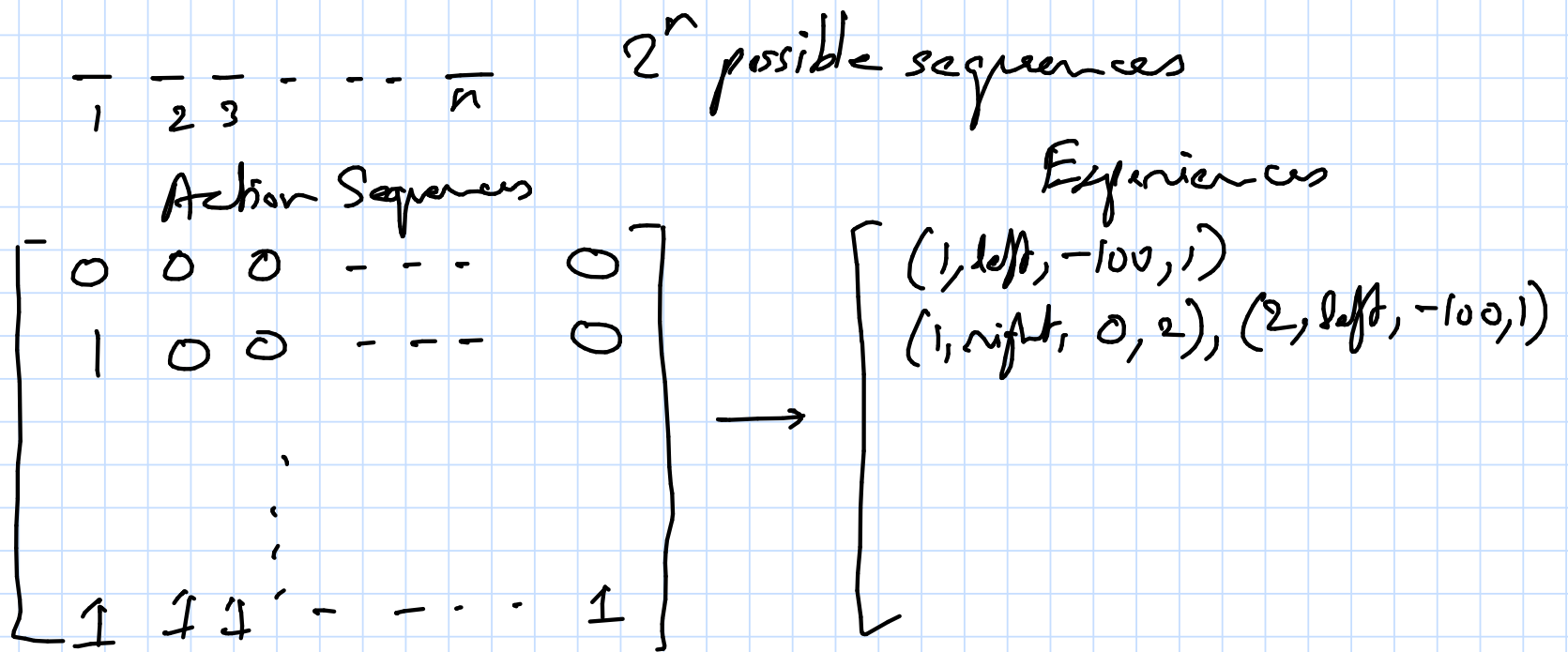
end for

end for

Motivation : Blind Cliffwalk



Example seq of actions (left=0, right=1)



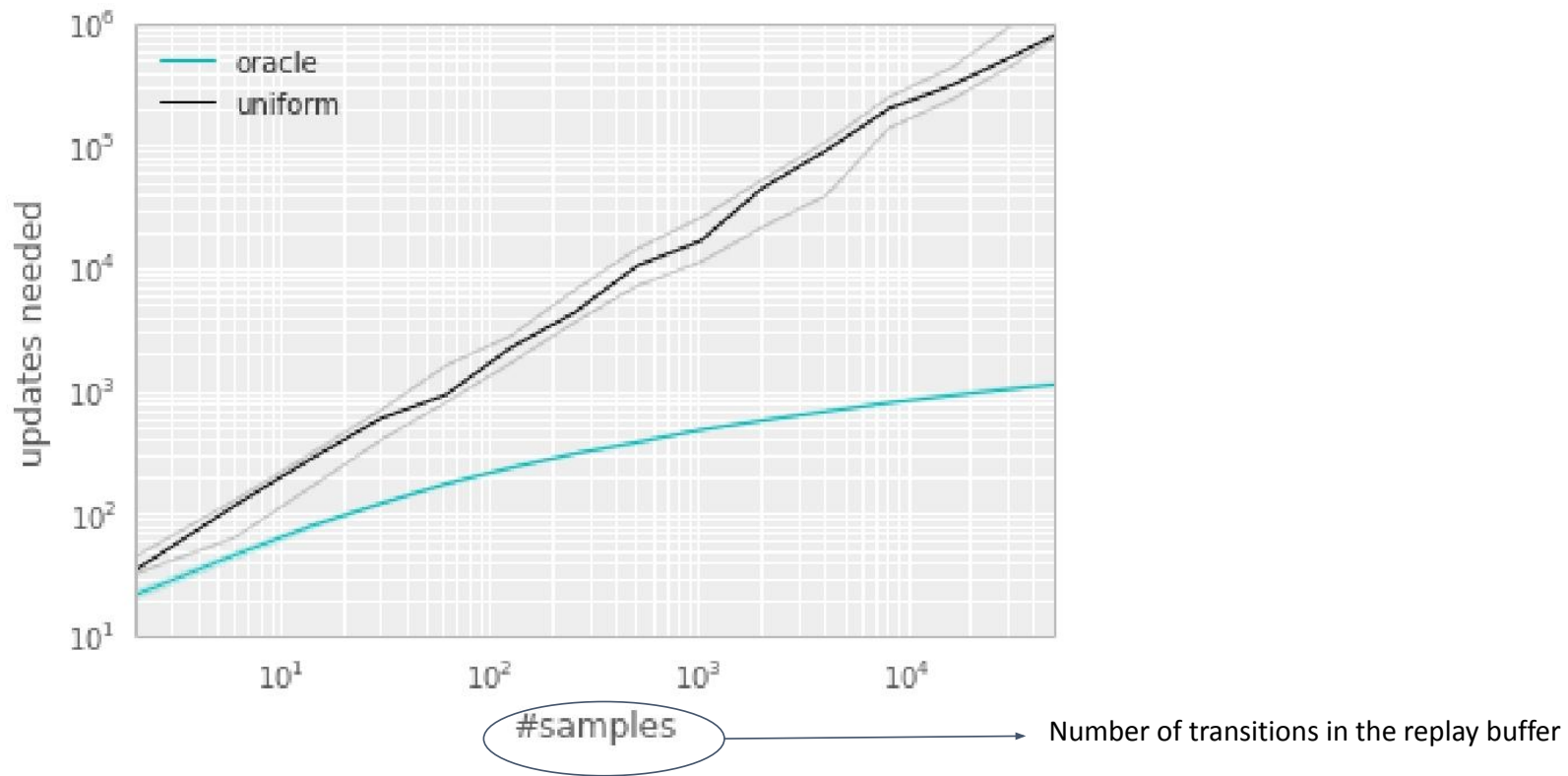
Blind Cliffwalk

- Generate all possible sequences ^{of actions} in a random order
 - Add resulting experiences to the replay buffer
 - Replay buffer has transitions with the same frequency as they occur when generating sequences
- ~~How many sequences?~~ On an average, how many sequences need to be generated to see a return of 1?
- How many unique transitions? (experiences in the replay buffer)
- Compare *Oracle* based replay with experiences chosen uniformly

Blind Cliffwalk

- Oracle calculates the parameter update (using Q-learning) corresponding to every transition
- Picks the transition (the corr parameter update) that minimizes the *loss function*
 - *seemingly* the average TD error over all transitions in the buffer

Gains Over Uniform Replay Motivate Prioritized Replay



Greedy TD-Error Based Prioritization

- Store last calculated TD-error with a transition in the replay buffer
- Pick transition that has largest TD-error for replay
- New transitions are replayed with the highest priority
 - Ensures that all transitions are played at least once
 - New transitions post replay get associated with a TD-error

Greedy TD-Error Based Prioritization

- Store last calculated TD-error with a transition in the replay buffer
- Pick transition that has largest (absolute value) TD-error for replay
- TD-errors may shrink slowly
 - You may end up replaying the high error transitions frequently
 - And not replaying the low error transitions for a while
- *Note that TD-errors are updated for a transition only when it is replayed*

Stochastic Prioritization

- Allow picking low TD-error transitions with non-zero probability
- Probability of sampling a transition is monotonically increasing in absolute value of its TD-error

Probability of Sampling a Transition i

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

Two Possibilities

- Proportional Prioritization

$$p_i = |\delta_i| + \epsilon$$

- Rank based Prioritization

$$p_i = \frac{1}{\text{rank}(i)}$$

Importance Sampling to Correct the Bias

- Prioritized replay changes the distribution of transitions as used to update weights
 - In comparison to the (target) distribution observed by the agent creating the experiences
- Compensate for the bias introduced by prioritized replay using an importance sampling factor
- Read about Annealing the Bias (hyperparameter β)
 - When do we care to apply the IS factor?

Algorithm

Algorithm 1 Double DQN with proportional prioritization

```

1: Input: minibatch  $k$ , step-size  $\eta$ , replay period  $K$  and size  $N$ , exponents  $\alpha$  and  $\beta$ , budget  $T$ .
2: Initialize replay memory  $\mathcal{H} = \emptyset$ ,  $\Delta = 0$ ,  $p_1 = 1$ 
3: Observe  $S_0$  and choose  $A_0 \sim \pi_\theta(S_0)$ 
4: for  $t = 1$  to  $T$  do
5:   Observe  $S_t, R_t, \gamma_t$ 
6:   Store transition  $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t)$  in  $\mathcal{H}$  with maximal priority  $p_t = \max_{i < t} p_i$ 
7:   if  $t \equiv 0 \pmod K$  then
8:     for  $j = 1$  to  $k$  do
9:       Sample transition  $j \sim P(j) = p_j^\alpha / \sum_i p_i^\alpha$ 
10:      Compute importance-sampling weight  $w_j = (N \cdot P(j))^{-\beta} / \max_i w_i$ 
11:      Compute TD-error  $\delta_j = R_j + \gamma_j Q_{\text{target}}(S_j, \arg \max_a Q(S_j, a)) - Q(S_{j-1}, A_{j-1})$ 
12:      Update transition priority  $p_j \leftarrow |\delta_j|$ 
13:      Accumulate weight-change  $\Delta \leftarrow \Delta + w_j \cdot \delta_j \cdot \nabla_\theta Q(S_{j-1}, A_{j-1})$ 
14:    end for
15:    Update weights  $\theta \leftarrow \theta + \eta \cdot \Delta$ , reset  $\Delta = 0$ 
16:    From time to time copy weights into target network  $\theta_{\text{target}} \leftarrow \theta$ 
17:  end if
18:  Choose action  $A_t \sim \pi_\theta(S_t)$ 
19: end for

```

Suppose $\beta = 1$

$$\frac{1}{N P(j)} = \frac{1/N}{P(j)} = \frac{1/N}{1/N} = 1$$

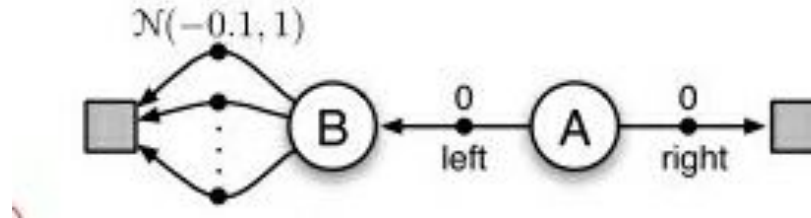
Double Deep Q Network

Deep Reinforcement Learning with Double Q-learning

Hado van Hasselt and Arthur Guez and David Silver
Google DeepMind

Recall Maximization Bias

Maximization Bias



- Suppose during an episode taking action left in State B gets a large positive reward
- Impacts action-value of (State A, action left)
- Together with greedy (maximizing) action selection may mean left will be chosen in state A for a long time
 - Till many steps later $Q(B, \text{left})$ gets updated to a value closer to average -0.1

Image: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>

Maximization Bias

- Agents actions are biased by our currently maximum action-value estimates
- May slow down reaching the true optimal action-value estimates
- Alleviate bias by maintaining more than one independent valuations of (s,a)
 - Choose your greedy action but when bootstrapping value it as per the other valuation

Double Q-learning

- Maintain independent Q -value tables $Q(s,a)$ and $Q'(s,a)$ for all (s,a)
- The updates for the independent tables are:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q'(S_{t+1}, \arg \max_{a \in A(S_{t+1})} Q(S_{t+1}, a)) - Q(S_t, A_t))$$

$$Q'(S_t, A_t) = Q'(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, \arg \max_{a \in A(S_{t+1})} Q'(S_{t+1}, a)) - Q'(S_t, A_t))$$

DDQN

- Don't maintain two Q-networks
- Simply maintain a separate copy of the weights to calculate the target
 - Update the target copy less frequently with the current weight vector

$$Y_t^{\text{DoubleDQN}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \underset{a}{\operatorname{argmax}} Q(S_{t+1}, a; \boldsymbol{\theta}_t), \boldsymbol{\theta}_t^-)$$

Dueling Network Architectures for Deep Reinforcement Learning

Ziyu Wang

Tom Schaul

Matteo Hessel

Hado van Hasselt

Marc Lanctot

Nando de Freitas

Google DeepMind, London, UK

ZIYU@GOOGLE.COM

SCHAUL@GOOGLE.COM

MTTHSS@GOOGLE.COM

HADO@GOOGLE.COM

LANCTOT@GOOGLE.COM

NANDODEFREITAS@GMAIL.COM

The Dueling Network

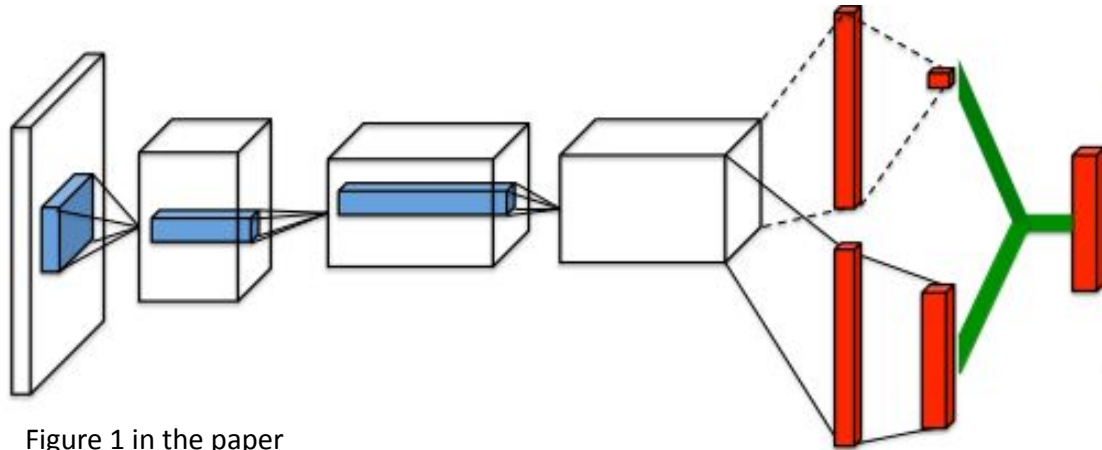


Figure 1 in the paper

- Learn separate representations of the state-value and state-action advantages
- Combine the value and advantage streams to produce action-value estimates for all actions of the input state

(Action) Advantage Function

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

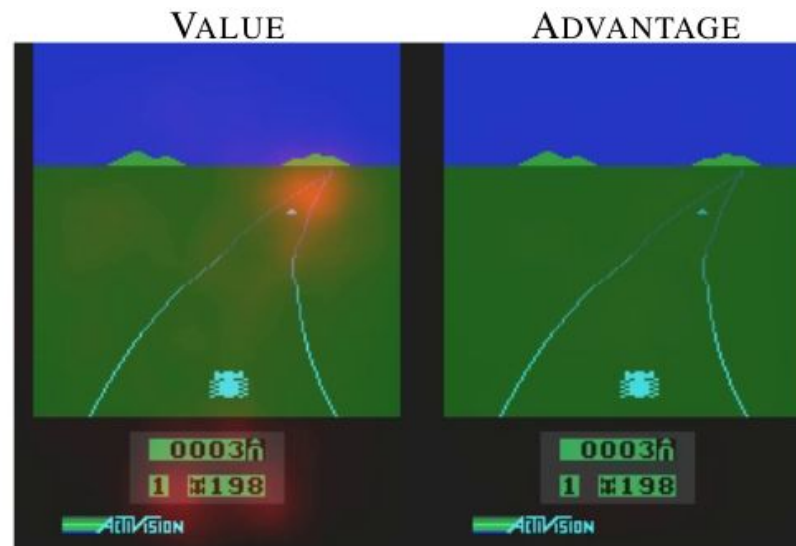
- What's the advantage of the optimal action for a state?

Motivation

- The choice of action may be inconsequential in many states
- The value of a state is important for bootstrapping (update value stream specific weights more often)
 - Q: But we used Q-values to bootstrap. How does value of a state help?

Motivation

- The choice of action may be inconsequential in many states
- Saliency Maps (Jacobian of the streams wrt the input)



Motivation

- The choice of action may be inconsequential in many states
- Saliency Maps (Jacobian of the streams wrt the input)

